Block Color Sorting

1. Function Description

After the program starts, the robotic arm will perform color recognition based on the set HSV values. After pressing the spacebar, the robotic arm will lower the gripper to grip the recognized block and place it at the set position. After placement is complete, it returns to the recognition pose.

2. Startup and Operation

2.1 Startup Commands

Enter the following commands in the terminal to start:

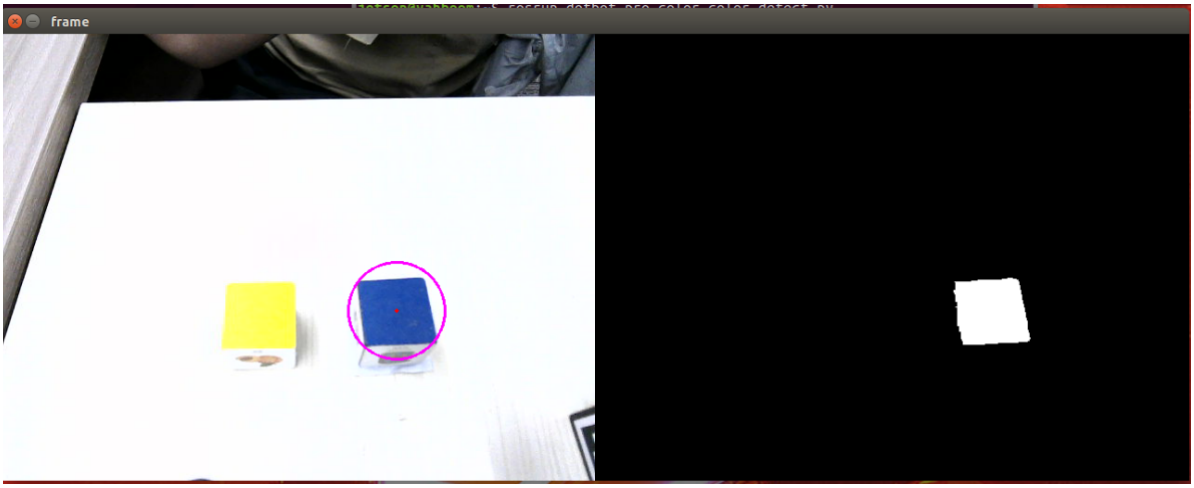
```
# Start camera, inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start color recognition program:
ros2 run dofbot_sorting_3d color_sorting
# Start robotic arm gripping program:
ros2 run dofbot_sorting_3d grasp
```

2.2 Operation Process

After the terminal starts, place blocks of different colors. The camera captures the image. Process the image with the following keys:

- 【i】or【I】: Enter color recognition mode, directly load the HSV values calibrated from the last program run for recognition;
- 【r】, 【g】, 【b】, 【y】: Select the block color to sort, representing red, green, blue, and yellow respectively
- 【c】: Exit color recognition mode, then you can press 【r】, 【g】, 【b】, 【y】 to select color again, corresponding to red, green, blue, yellow
- Spacebar: Start gripping the recognized block (generally only needs to be pressed once, subsequent blocks will be recognized automatically)

As shown in the image below, when the left image (binarized) shows only one uniquely recognized color, click on the color image frame and press the spacebar. The robotic arm will lower the gripper to grip the recognized block.



3. Program Flowchart

```
flowchart TB
    A[Start] --> B[Program initialization including creating nodes, publishers
and subscribers]
    B --> C[Receive color image topic, enter callback function]
    C --> D[Process received image message, return processed color image and
binarized image, return circle parameters]
    D --> E{Check if self.cx, self.cy, self.circle_r meet conditions}
    E -- N --> D
    E -- Y --> F[Conditions met, check if self.cx, self.cy exceed limits, if not,
assign message data, check if self.pubPos_flag is true, then publish block
information topic message]
    F --> G{Check if binarized image exists}
    G -- Y --> H[Exists, display color image and binarized image]
    G -- N --> I[Does not exist, only display color image]
```

4. Core Code Analysis

color_sorting.py

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/color_sorting.py
```

Import necessary libraries,

```
from dofbot_sorting_3d.compute_joint5 import *
import cv2
import os
import numpy as np
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2 as cv
from dofbot_sorting_3d.color_common import *
from dofbot_interface.srv import Kinemarics
from dofbot_interface.msg import *
from std_msgs.msg import Bool, Int16
import time
from rclpy.node import Node
import rclpy
import threading
from Arm_Lib import Arm_Device
package_pwd = "/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/"
#Set exposure parameters
exit_code = os.system('sudo v4l2-ctl -d /dev/video0 -c brightness=10')
```

Initialize program parameters, create publishers, subscribers, etc.

```
def __init__(self, name):
    # Initialize node Initialize node
    super().__init__(name)
    # Create arm device object Create arm device object
    self.Arm = Arm_Device()
    # Initial joint angles Initial joint angles
```

```
    self.init_joints = [90, 120, 0, 0, 90, 0]
    # Create RGB image bridge object Create RGB image bridge object
    self.rgb_bridge = CvBridge()
    # Position publish flag Position publish flag
    self.pub_pos_flag = False
    # Create grasp completion status subscriber Create grasp completion status
subscriber
    self.sub_grasp_status = self.create_subscription(
        Bool, "grasp_done", self.get_graspStatusCallBack, 100)
    # Create AprilTag position info publisher Create AprilTag position info
publisher
    self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo", 1)
    # Create joint5 angle publisher Create joint5 angle publisher
    self.pub_joint5 = self.create_publisher(Int16, "set_joint5", 10)
    # Target color Target color
    self.target_color = 0
    # Red HSV threshold file path Red HSV threshold file path
    self.red_hsv_text = os.path.join(package_pwd, 'red_colorHSV.text')
    # Green HSV threshold file path Green HSV threshold file path
    self.green_hsv_text = os.path.join(package_pwd, 'green_colorHSV.text')
    # Blue HSV threshold file path Blue HSV threshold file path
    self.blue_hsv_text = os.path.join(package_pwd, 'blue_colorHSV.text')
    # Yellow HSV threshold file path Yellow HSV threshold file path
    self.yellow_hsv_text = os.path.join(
        package_pwd, 'yellow_colorHSV.text')
    # HSV color range HSV color range
    self.hsv_range = ()
    # Selection flag Selection flag
    self.select_flags = False
    # Window name Window name
    self.windows_name = 'frame'
    # Track state Track state
    self.Track_state = 'init'
    # Mouse coordinates Mouse coordinates
    self.Mouse_XY = (0, 0)
    # Image columns and rows Image columns and rows
    self.cols, self.rows = 0, 0
    # Region of interest initialization Region of interest initialization
    self.Roi_init = ()
    # Color detection object Color detection object
    self.color = color_detect()
    # Current color Current color
    self.cur_color = None
    # Text color Text color
    self.text_color = (0, 0, 0)
    # Circle center X coordinate Circle center X coordinate
    self.cx = 0
    # Circle center Y coordinate Circle center Y coordinate
    self.cy = 0
    # Circle radius Circle radius
    self.circle_r = 0
    # Joint angle message Joint angle message
    self.joint5 = Int16()
    # Corner array Corner array
    self.corners = np.empty((4, 2), dtype=np.int32)
    # Current target color Current target color
    self.cur_target_color = 0
    # Update flag Update flag
```

```
    self.updata_flag = False
    # Set arm initial joint angles Set arm initial joint angles
    self.Arm.Arm_serial_servo_write6_array(self.init_joints, 2000)
    # Target distance Target distance
    self.dist = 0.13
    # Create image subscriber Create image subscriber
    self.subscription = self.create_subscription(
        Image, '/image_raw', self.ImageCallback, 1)
```

Let's look at the main image processing function ImageCallback

```
def ImageCallback(self, color_frame):
    # Convert ROS image message to OpenCV format Convert ROS image message to
OpenCV format
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')
    # Convert RGB image to BGR format Convert RGB image to BGR format
    rgb_image = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)
    # Copy image for display Copy image for display
    result_image = np.copy(rgb_image)
    # Detect key input Detect key input
    key = cv2.waitKey(10) & 0xFF
    # Process image and get result frame and binary image Process image and get
result frame and binary image
    result_frame, binary = self.process(rgb_image, key)
    # Create thread for displaying image Create thread for displaying image
    show_frame = threading.Thread(
        target=self.img_out, args=(result_frame, binary,))
    # Start display thread Start display thread
    show_frame.start()
    # Wait for display thread to finish Wait for display thread to finish
    show_frame.join()
    # If space key (ASCII 32) is pressed, set publish flag If space key (ASCII
32) is pressed, set publish flag
    if key == 32:
        self.pub_pos_flag = True
    # If valid target is detected (center and radius are valid, and target color
is not 0)
    # If valid target is detected (center and radius are valid, and target color
is not 0)
    if self.cx != 0 and self.cy != 0 and self.circle_r > 30 and
self.target_color != 0:
        # Convert circle center coordinates to integers Convert circle center
coordinates to integers
        cx = int(self.cx)
        cy = int(self.cy)
        # Calculate target position in arm coordinate system
        # Calculate target position in arm coordinate system
        (a, b) = (round(((320 - cx)/4000), 5),
                  round(((480 - cy)/3000) * 0.8+0.13, 5))
        # Calculate corner vector difference Calculate corner vector difference
        vx = self.corners[0][0][0] - self.corners[1][0][0]
        vy = self.corners[0][0][1] - self.corners[1][0][1]
        # Calculate target angle for joint Calculate target angle for joint
        target_joint5 = compute_joint5(vx, vy)

        # If publish flag is true, publish position information  If publish flag
is true, publish position information
```

```
        if self.pub_pos_flag == True:
            # Reset publish flag  Reset publish flag
            self.pub_pos_flag = False
            # Create joint5 angle message  Create joint5 angle message
            joint5 = Int16()
            # Set joint angle data  Set joint angle data
            joint5.data = int(target_joint5)
            # Publish joint angle  Publish joint angle
            self.pub_joint5.publish(joint5)
            # Create AprilTag position info message Create AprilTag position
info message
            pos = AprilTagInfo()
            # Set color ID (1=blue, 2=green, 3=red, 4=yellow)
            # Set color ID (1=blue, 2=green, 3=red, 4=yellow)
            pos.id = self.target_color
            # Set coordinates
            pos.x = float(b)
            pos.y = float(a)
            pos.z = 0.03
            # Publish position information  Publish position information
            self.pos_info_pub.publish(pos)

            color_names = {0: "Unknown", 1: "Blue",
                           2: "Green", 3: "Red", 4: "Yellow"}

        else:
            # When space key is not pressed, only output detected color info
without publishing position
            # When space key is not pressed, only output detected color info
without publishing position
            print(...)
```

Image processing function self.process,

```
def process(self, rgb_img, key):
    # Resize image to 640x480
    rgb_img = cv.resize(rgb_img, (640, 480))
    # Initialize binary image list  Initialize binary image list
    binary = []

    # Press C key: clear target color and reset  Press C key: clear target color
and reset
    if key == ord('c') or key == ord('C'):
        self.target_color = 0
        self.Reset()
        self.updata_flag = True
    # Press B key: set target color to blue  Press B key: set target color to
blue
    elif key == ord('b') or key == ord('B'):
        self.target_color = 1
        self.cur_target_color = self.target_color
    # Press G key: set target color to green  Press G key: set target color to
green
    elif key == ord('g') or key == ord('G'):
        self.target_color = 2
        self.cur_target_color = self.target_color
    # Press R key: set target color to red  Press R key: set target color to red
```

```
    elif key == ord('r') or key == ord('R'):
        self.target_color = 3
        self.cur_target_color = self.target_color
    # Press Y key: set target color to yellow Press Y key: set target color to
yellow
    elif key == ord('y') or key == ord('Y'):
        self.target_color = 4
        self.cur_target_color = self.target_color
    # Press I key or target color exists: enter identify mode Press I key or
target color exists: enter identify mode
    elif key == ord('i') or key == ord('I') or self.target_color != 0:
        self.Track_state = "identify"
    # Print track state for debugging Print track state for debugging
    # print("self.Track_state: ",self.Track_state)
    # Init state: create window and set mouse callback Init state: create window
and set mouse callback
    if self.Track_state == 'init':
        cv.namedWindow(self.windows_name, cv.WINDOW_AUTOSIZE)
        cv.setMouseCallback(self.windows_name, self.onMouse, 0)
        # If region is selected, draw selection box If region is selected, draw
selection box
        if self.select_flags == True:
            cv.line(rgb_img, self.cols, self.rows, (255, 0, 0), 2)
            cv.rectangle(rgb_img, self.cols, self.rows, (0, 255, 0), 2)
            # If selection region is valid, calculate HSV range If selection
region is valid, calculate HSV range
            if self.Roi_init[0] != self.Roi_init[2] and self.Roi_init[1] !=
self.Roi_init[3]:
                rgb_img, self.hsv_range = self.color.Roi_hsv(
                    rgb_img, self.Roi_init)
                self.dyn_update = True
            else:
                self.Track_state = 'init'

    # Identify state: load HSV threshold based on target color Identify state:
load HSV threshold based on target color
    elif self.Track_state == "identify":
        # Blue: load blue HSV threshold Blue: load blue HSV threshold
        if self.target_color == 1:
            self.hsv_range = read_HSV(self.blue_hsv_text)
            self.cur_color = "blue"
            self.text_color = (255, 0, 0)

        # Green: load green HSV threshold Green: load green HSV threshold
        elif self.target_color == 2:
            self.hsv_range = read_HSV(self.green_hsv_text)
            self.cur_color = "green"
            self.text_color = (0, 255, 0)

        # Red: load red HSV threshold Red: load red HSV threshold
        elif self.target_color == 3:
            self.hsv_range = read_HSV(self.red_hsv_text)
            self.cur_color = "red"
            self.text_color = (0, 0, 255)

        # Yellow: load yellow HSV threshold Yellow: load yellow HSV threshold
        elif self.target_color == 4:
            self.hsv_range = read_HSV(self.yellow_hsv_text)
```

```
            self.cur_color = "yellow"
            self.text_color = (255, 255, 0)

        # Invalid color: return to init state Invalid color: return to init
state
        else:
            self.Track_state = 'init'

    if self.Track_state != 'init':
        # If HSV range is valid, perform object tracking If HSV range is valid,
perform object tracking
        if len(self.hsv_range) != 0:
            rgb_img, binary, self.circle, _, self.corners =
self.color.object_follow(
                rgb_img, self.hsv_range)
            # Save circle center coordinates Save circle center coordinates
            self.cx = self.circle[0]
            self.cy = self.circle[1]
            # Save circle radius Save circle radius
            self.circle_r = self.circle[2]
            # Save HSV threshold to file based on current target color Save HSV
threshold to file based on current target color
            if self.cur_target_color == 1 and self.updata_flag == True:
                write_HSV(self.blue_hsv_text, self.hsv_range)
            elif self.cur_target_color == 2 and self.updata_flag == True:
                write_HSV(self.green_hsv_text, self.hsv_range)
            elif self.cur_target_color == 3 and self.updata_flag == True:
                write_HSV(self.red_hsv_text, self.hsv_range)
            elif self.cur_target_color == 4 and self.updata_flag == True:
                write_HSV(self.yellow_hsv_text, self.hsv_range)
            # Reset update flag Reset update flag
            self.updata_flag = False

    # Draw current color name on image Draw current color name on image
    rgb_img = cv2.putText(rgb_img, self.cur_color, (10, 30),
                          cv2.FONT_HERSHEY_SIMPLEX, 1, self.text_color, 2)
    # Return processed image and binary image Return processed image and binary
image
    return rgb_img, binary
```

grasp.py

The previous chapter has explained this, so I won't repeat it here